

Escuela Politécnica Nacional
Facultad de Ingeniería en Sistemas

INFORME TÉCNICO: IMPLEMENTACIÓN DE ANALIZADORES LÉXICOS EN LEX.

Asignatura: Compiladores y Lenguajes

Autores: Aidan Carrasco, Felipe Merino y Mathew Verdezoto

1. Resumen

En este ejercicio implementaremos un analizador léxico que procesará una lista de elementos separados por comas, clasificándolos estrictamente como identificadores básicos o números enteros.

2. Estructura de un Archivo LEX

Un archivo fuente lex se encuentra estructurado por los siguientes bloques funcionales separados por (%%):

- **Área de declaración de variables:** Se declaran las librerías y variables que se van a utilizar.
- **Área de expresiones regulares:** En esta se adjuntan todas las expresiones regulares necesarias para cada programa y su respectiva impresión en consola si se llega a encontrar algo relacionado con la misma.
- **Área de código:** En esta se empieza a codificar en el lenguaje previamente seleccionado.

3. Implementación y Código Fuente

```
iccd332@DESKTOP-52QQFVV x + v
(base) iccd332@DESKTOP-52QQFVV:~$ mkdir veinte
(base) iccd332@DESKTOP-52QQFVV:~$ cd veinte
(base) iccd332@DESKTOP-52QQFVV:~/veinte$ nano veinte.l
(base) iccd332@DESKTOP-52QQFVV:~/veinte$ nano veinte.l
(base) iccd332@DESKTOP-52QQFVV:~/veinte$ nano entrada.txt
(base) iccd332@DESKTOP-52QQFVV:~/veinte$ lex veinte.l
(base) iccd332@DESKTOP-52QQFVV:~/veinte$ gcc lex.yy.c -lfl
(base) iccd332@DESKTOP-52QQFVV:~/veinte$ ./a.out < entrada.txt > salida.
txt
(base) iccd332@DESKTOP-52QQFVV:~/veinte$ cat salida.txt
Elemento: IDENTIFICADOR (variable1)
--- COMA ---
Elemento: NUMERO ENTERO (2026)
--- COMA ---
Elemento: IDENTIFICADOR (suma)
--- COMA ---
Elemento: NUMERO ENTERO (45)
--- COMA ---
ERROR: Tipo de token no reconocido ('@')
--- COMA ---
Elemento: IDENTIFICADOR (identificador2)
--- COMA ---
ERROR: Tipo de token no reconocido ('$')
--- COMA ---
Elemento: NUMERO ENTERO (99)
```

```
iccd332@DESKTOP-52QQFVW x + v
GNU nano 7.2 veinte.l
%{
#include <stdio.h>
}%
%%
[a-zA-Z][a-zA-Z0-9]* { printf("Elemento: IDENTIFICADOR (%s)\n", yytext); }
[0-9]+ { printf("Elemento: NUMERO ENTERO (%s)\n", yytext); }
", " { printf("--- COMA ---\n"); }
[ \t\n]+
. { printf("ERROR: Tipo de token no reconocido ('%s)\n", yytext); }
%%
int main()
{
    yylex();
    return 0;
}
```

```
iccd332@DESKTOP-52QQFVW x + v
GNU nano 7.2 entrada.txt
variable1, 2026, suma, 45, @, identificador2, $, 99
```

```
salida.txt
Archivo Editar Ver H1 v ≡ v B I

Elemento: IDENTIFICADOR (variable1)
--- COMA ---
Elemento: NUMERO ENTERO (2026)
--- COMA ---
Elemento: IDENTIFICADOR (suma)
--- COMA ---
Elemento: NUMERO ENTERO (45)
--- COMA ---
ERROR: Tipo de token no reconocido ('@')
--- COMA ---
Elemento: IDENTIFICADOR (identificador2)
--- COMA ---
ERROR: Tipo de token no reconocido ('$')
--- COMA ---
Elemento: NUMERO ENTERO (99)
```

3. Conclusión

Esta práctica demuestra la utilidad de Lex para la clasificación de tokens y el manejo proactivo de errores léxicos mediante la regla por defecto. Además, evidencia una limitación clave del análisis léxico.